Mall Delivery App: Functional Prototype

React Native + Node.js + PostgreSQL + Socket.io
Goal: Real-time mall delivery with Try & Buy.

Objective: Four actors interact live with state changes.
Actor Legend:

🟩 Customer

🟦 Store

🟧 Mall Runner

🟪 Delivery Rider

System Architecture

Frontend: React Native mobile app; React web Store Dashboard
Backend: Node.js, Express API, Socket.io realtime
Database: PostgreSQL, Prisma ORM
External: Supabase, Cloudinary, Railway, GitHub/Vercel
Data Flow:

- 🟩 Customer → API → DB
- 🟦 Store Dashboard ↔ Socket.io
- 🟧/🟪 Runner/Rider ↔ API
- Realtime events via Socket.io rooms

Order Lifecycle

Forward: PENDING → RUNNER_ASSIGNED → COLLECTED → HANDED_TO_RIDER → OUT_FOR_DELIVERY → ARRIVED → DELIVERED

Return: ARRIVED → RETURNING → RETURNED → REFUNDED

Ownership: PENDING (🟩) → RUNNER_ASSIGNED/COLLECTED/HANDED_TO_RIDER (🟧) → OUT_FOR_DELIVERY/ARRIVED (🟪) → DELIVERED/RETURNING (🟩)

Customer Journey

Screens: Home → Search → Product Detail → Cart → Checkout → Order Confirm → Live Tracking → Try & Buy (10-min timer)

Emotions: Discovery → Intent → Anticipation → Excitement → Decision

Store Dashboard Workflow

Realtime: New order (socket) → Accept → Prepare → Runner arrival → Hand over → Mark picked

Features: Realtime updates, inventory, settlements, queue management

Runner Operational Flow

Accept → Go to store → Collect → Move to gate → Handover to rider

KPIs: Pickup time, handover speed, orders completed, store-to-gate time

Rider Delivery Flow

Accept → Pickup from runner → Navigate to customer → Live GPS → Try & Buy → Keep or Return

Features: GPS, timer facilitation, returns, payment collection

Database / Data Model

Entities: User (roles: ), Store, Product, Variant, Order, OrderItem

Relations: Customer places Orders; Store owns Products; Orders contain OrderItems; Products have Variants; Orders assigned to Runner/Rider

API Endpoints Map

Auth: POST /auth/register, POST /auth/login, GET /auth/me

Products: GET /products, GET /products/:id

Orders: POST /orders, PUT /orders/:id/status

Store: GET /store/orders, PUT /store/inventory

Runner/Rider: GET /runner|rider/orders, PUT /runner/orders/:id/collect, PUT /rider/orders/:id/deliver

Socket Events

Events: new_order, runner_order_available, rider_order_available, order_update, rider_location, try_timer_start, try_timer_end

Rooms: store:{id}, user:{id}, order:{id}, rider:{id}

Sprint 1 Kanban

Backlog: Payments, Reviews, Analytics

In Progress: Socket.io, Expo, Product screens

Review: Auth, Order lifecycle API

Done: Express+Prisma, DB schema, Git workflow

Cards: Setup, Auth, Order API, Sockets, Expo, Product, Checkout, Store realtime

Tech Stack

Layer	Technology
Frontend	React Native, Expo, NativeWind
Backend	Node.js, Express, Prisma
Database	PostgreSQL
Realtime	Socket.io
Hosting	Supabase, Railway, Vercel
Media	Cloudinary
VCS	GitHub

Team Workflow & Conventions

Git: Feature branch → PR → Review → Merge

Naming: PascalCase (components), camelCase (vars), snake_case (socket events)

Branches: feature/..., fix/..., refactor/...

Final Demo Flow

Act 1: 🟩 Order → 🟦 Realtime notify
Act 2: 🟦 Prepare → 🟧 Collect
Act 3: 🟧 Gate handoff → 🟪 Pickup
Act 4: 🟪 Delivery + GPS
Act 5: 🟩 Try & Buy (10 min)

Act 6: Keep (payment) or Return (refund)

Success Criteria

All actors functional; realtime updates; full lifecycle; timer works; live GPS; return/refund flow operational.